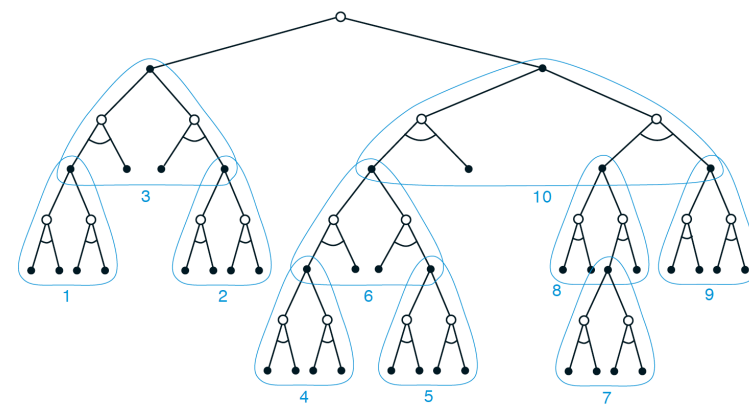
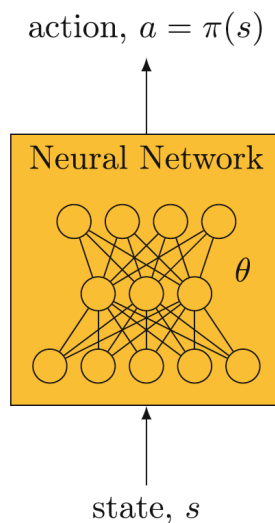
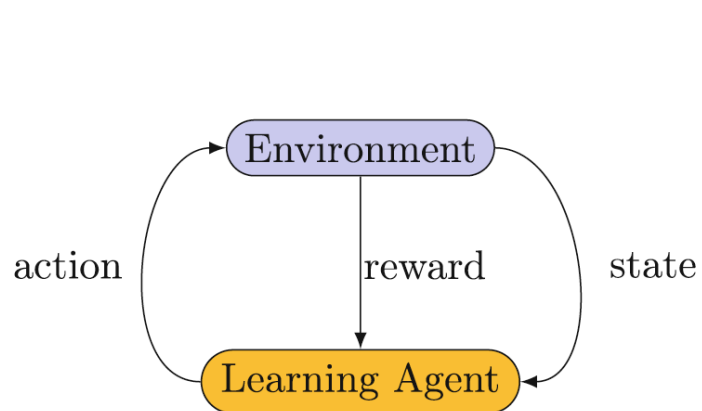


Deep Approximate Value Iteration and Deep Q-Learning

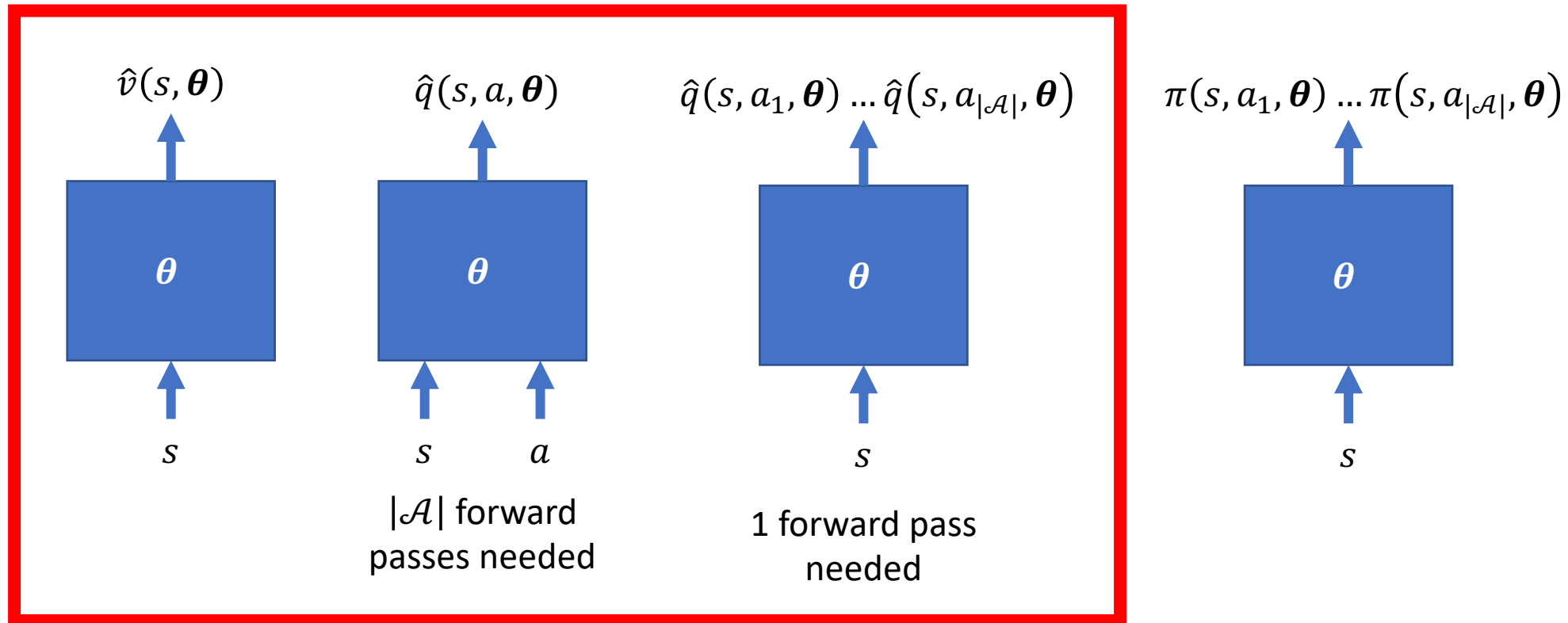
Forest Agostinelli
University of South Carolina



Outline

- Architecture and state representation
- Deep approximate value iteration (DAVI)
- Deep Q-networks (DQNs)

RL with Function Approximation

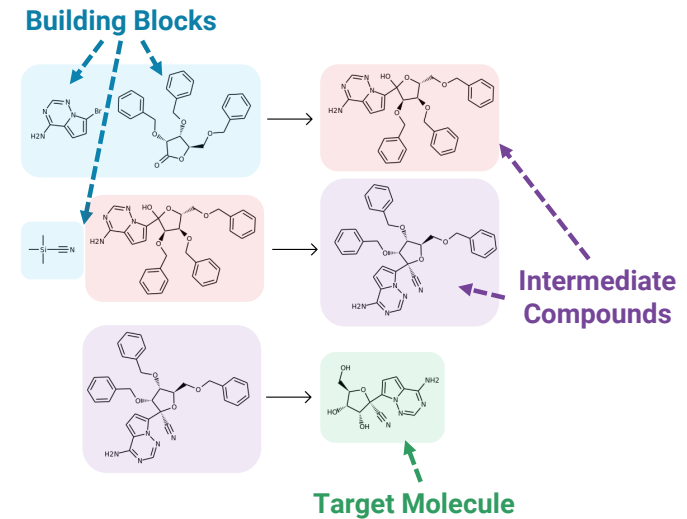
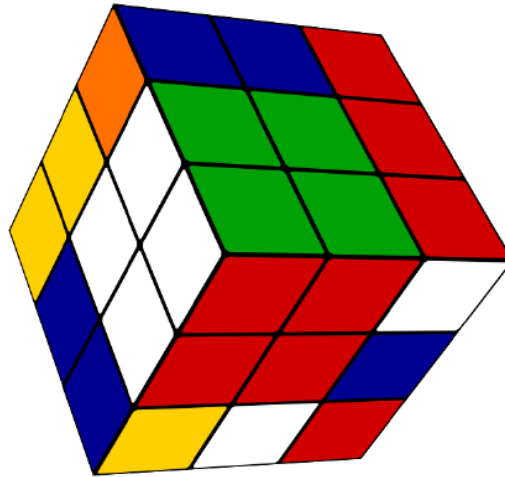
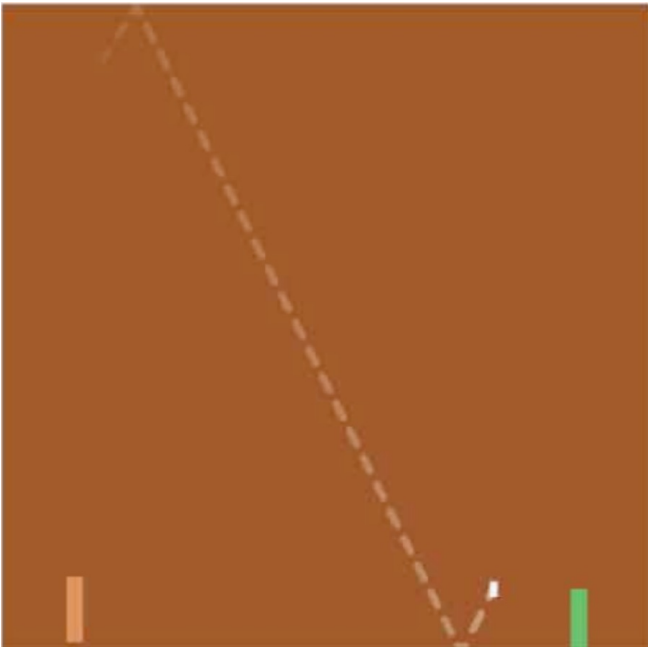


State Representation

- Since we cannot use a lookup table, we must translate our state to a numerical representation to be used by a deep neural network
- This representation can be structured
- There are usually multiple options for representing a state
- The state representation affects the type of neural network used

State Representation

- What type of state representation to use?
- What type of neural network to use?

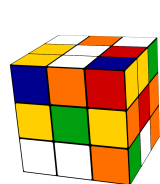


Outline

- Architecture and state representation
- Deep approximate value iteration (DAVI)
- Deep Q-networks (DQNs)

Approximate Value Iteration

- Sometimes, we can accurately define the MDP, however, the state space is too large to store in a table
- Therefore, we will need to use an architecture to approximate the value
- The goal is to have an architecture whose number of parameters is far less than the size of the state space

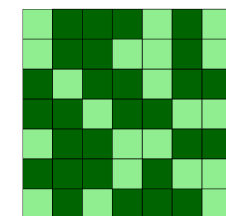


Rubik's cube

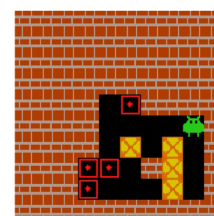
22	12	4	2	5
17	16	3	6	9
20	19	18	11	7
23	1		24	13
21	14	10	8	15

24 puzzle

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	

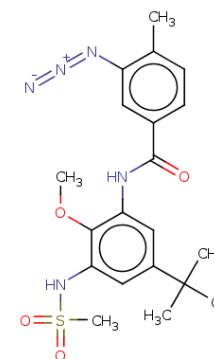


Lights Out (7x7)

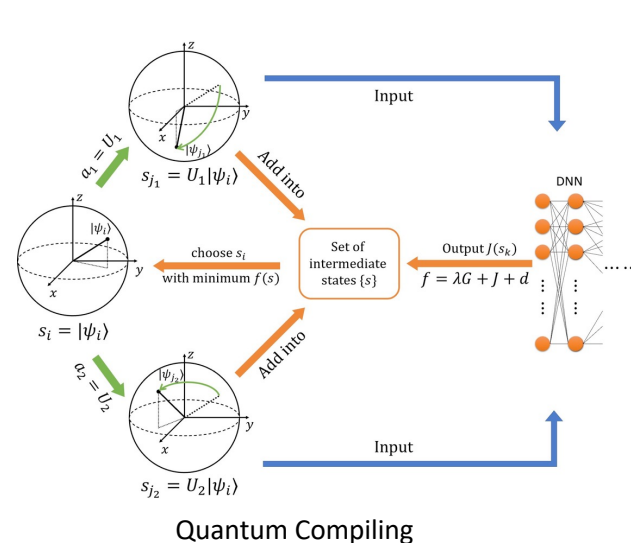


Sokoban

Puzzles



Chemical Synthesis



Approximate Value Iteration

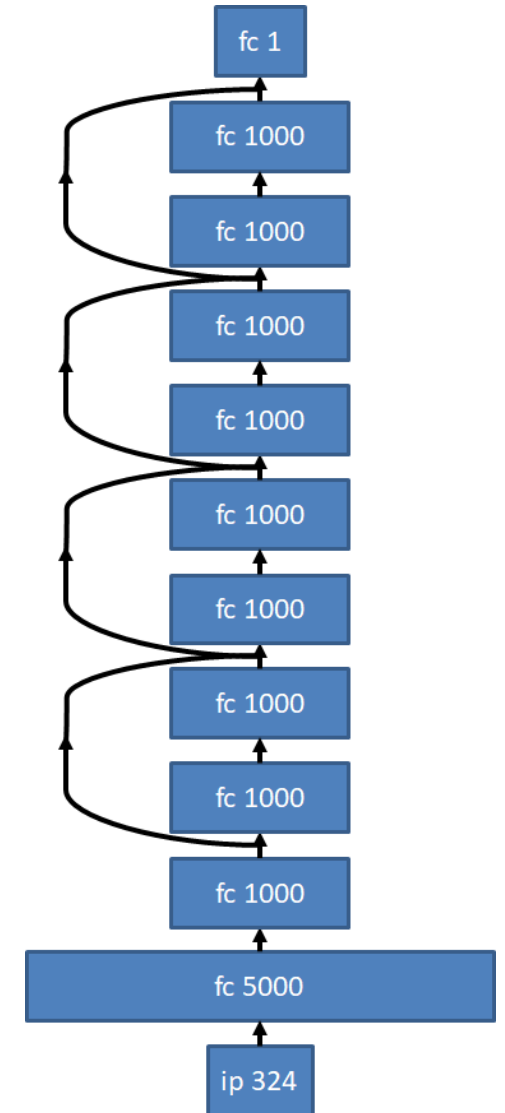
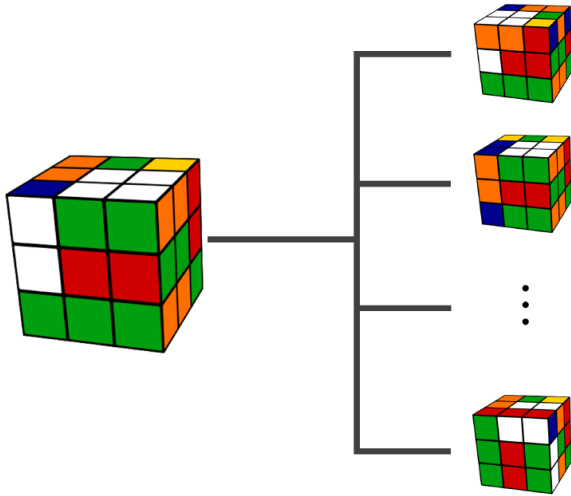
- Tabular Value Iteration
 - $V(s) = \max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s'))$
- Instead, we train a function approximator with parameters θ, v_θ
 - We will focus on doing this with gradient descent
- For a given state, s , we can compute the desired output, y , for randomly initialized parameters θ
 - $y =$
 - $\max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_\theta(s'))$ if s is not a terminal state
 - 0 otherwise
 - $L(\theta) = \frac{1}{2} (y - v_\theta(s))^2$
 - $\nabla_\theta L(\theta) = (y - v_\theta(s)) \nabla_\theta v_\theta(s)$
 - Even though y depends on θ , we do not differentiate y with respect to θ
- We can compute the loss using multiple states to get a better estimate of the gradient

Approximate Value Iteration: Convergence

- Update
 - $y =$
 - $\max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_\theta(s'))$ if s is not a terminal state
 - 0 otherwise
 - $L(\theta) = \frac{1}{2} (y - v_\theta(s))^2$
 - $\nabla_\theta L(\theta) = (y - v_\theta(s)) \nabla_\theta v_\theta(s)$
- Each time v_θ is updated there will be some inaccuracies
- These inaccuracies will then be present in the next updates
- The learned $v_\theta(s)$ approximates v_* with these accumulated inaccuracies
 - It is possible that inaccuracies are corrected by continuing to run approximate value iteration, but not guaranteed

Case Study: Rubik's Cube

- $y =$
 - $\max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_{\theta}(s'))$ if s is not a terminal state
 - 0 otherwise
- $L(\theta) = \frac{1}{2N} \sum_{i=1}^N (y_i - v_{\theta}(s_i))^2$
- Reward of -1 for every transition

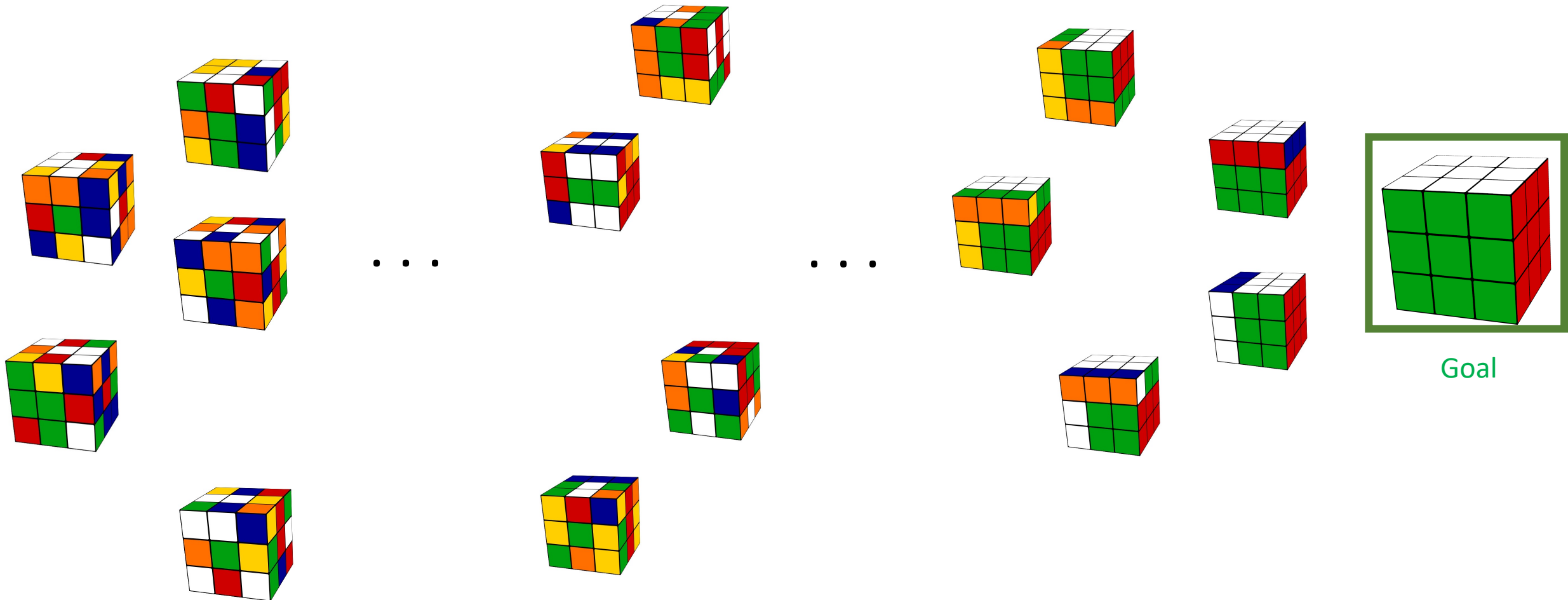


Data Generation

- If we start from a randomly sampled Rubik's cube state, it is very unlikely that a randomly initialized policy will find the terminal state
- Therefore, the learned value will continue to decrease

Prioritized Sweeping

- Prioritized sweeping: Generate training data by taking moves in reverse from the **goal**

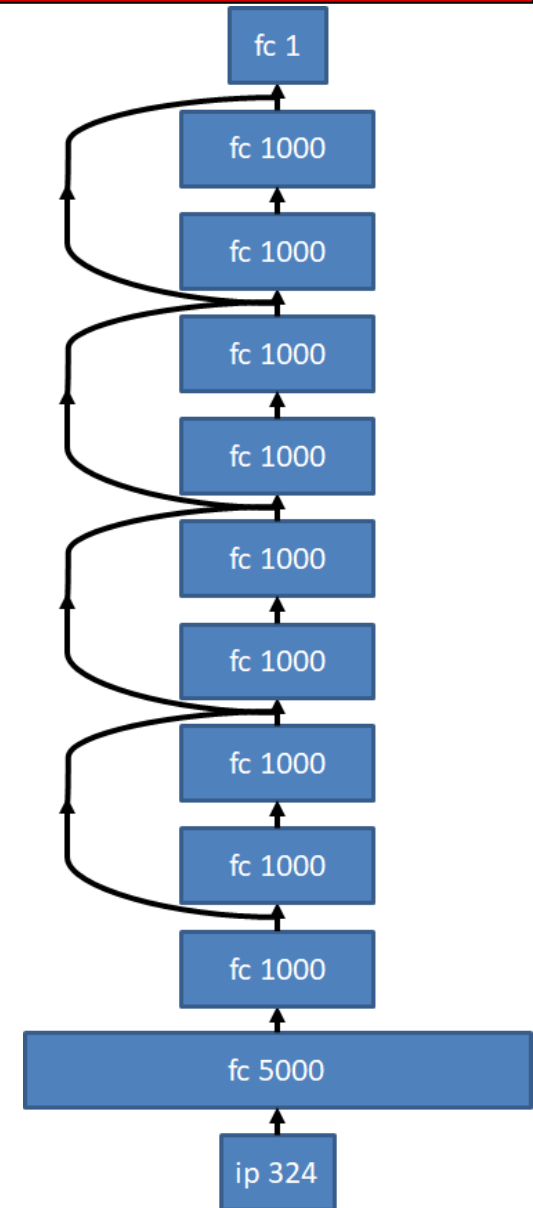


Non-Stationary Target

- Since the target is determined by v_θ , and v_θ changes every training iteration, the learned target also changes every training iteration
- Instead, we can maintain two sets of parameters, θ and θ^-
 - $y =$
 - $\max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_{\theta^-}(s'))$ if s is not a terminal state
 - 0 otherwise
 - $L(\theta) = \frac{1}{2} (y - v_\theta(s))^2$
 - $\nabla_\theta L(\theta) = (y - v_\theta(s)) \nabla_\theta v_\theta(s)$
- v_{θ^-} is known as the target network and θ^- is periodically updated to θ , allowing for periods of stationarity

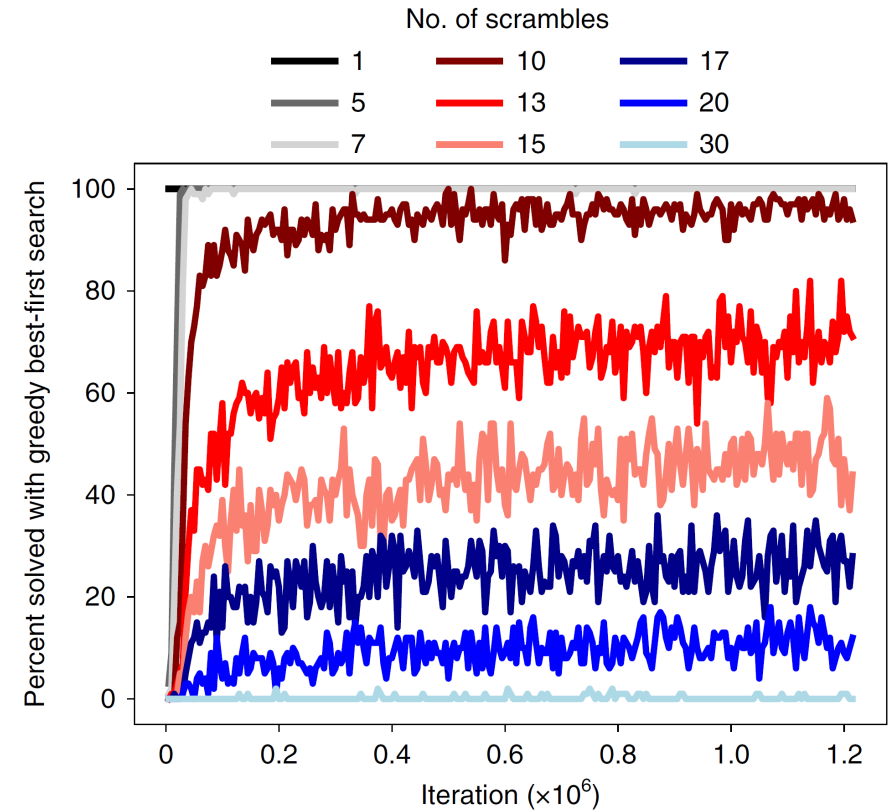
Training

- Deep neural network
 - Input layer -> Two fully connected layers -> Four residual blocks -> Linear output layer
- Training
 - Batch size of 5,000
 - ~1,000,000 training iterations
 - Parameters for target network updated when loss goes below some target threshold



Greedy Policy Performance

- Behave greedily with respect to the heuristic function
- $\pi(s) = \underset{a}{\operatorname{argmin}}(c^a(s) + h_\theta(T(s, a)))$
- Does not solve all states
 - Is this due to insufficiencies in reinforcement learning or network architecture/training?



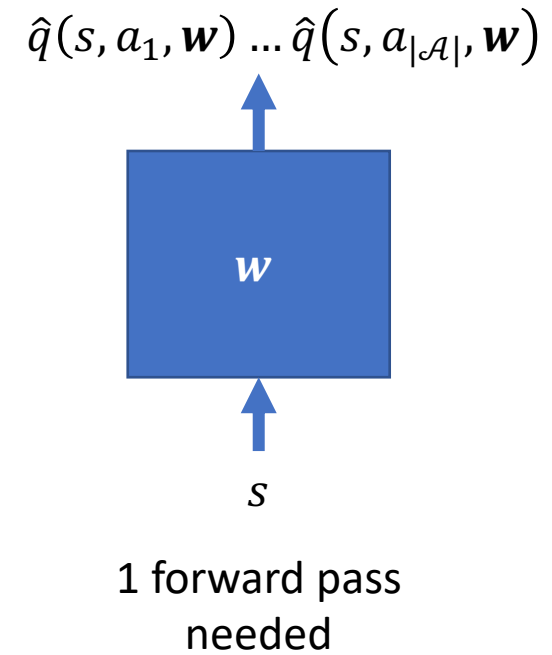
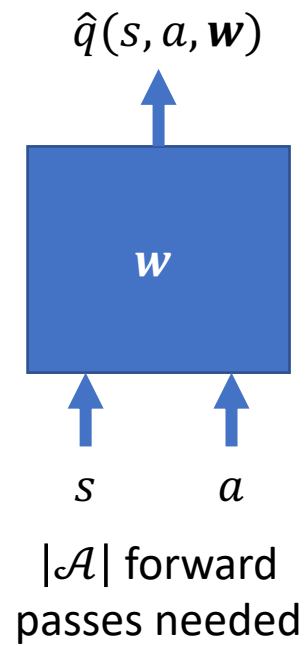
Outline

- Architecture and state representation
- Deep approximate value iteration (DAVI)
- Deep Q-networks (DQNs)

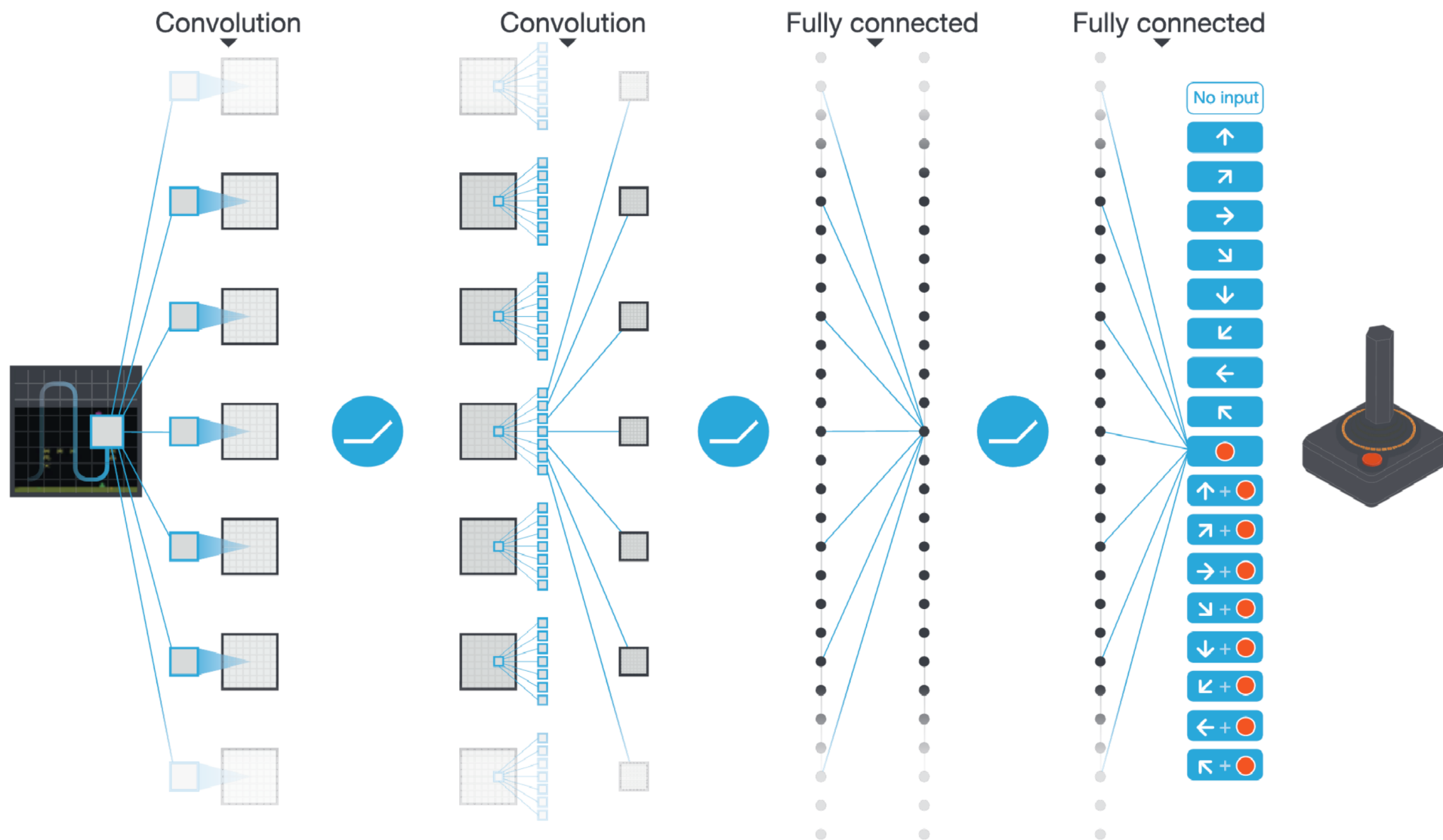
Approximate Q-learning

- Q-learning
 - $Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
- Approximate Q-learning
 - $y =$
 - $r + \gamma \max_{a'} q_{\theta}(s', a')$ if s is not a terminal state
 - 0 otherwise
 - $L(\theta) = \frac{1}{2} (y - q_{\theta}(s, a))^2$
 - $\nabla_{\theta} E(\theta) = (y - q_{\theta}(s, a)) \nabla_{\theta} q_{\theta}(s, a)$

Deep Q-Networks (DQNs)



DQNs: Atari



DQN: Issues that Arise

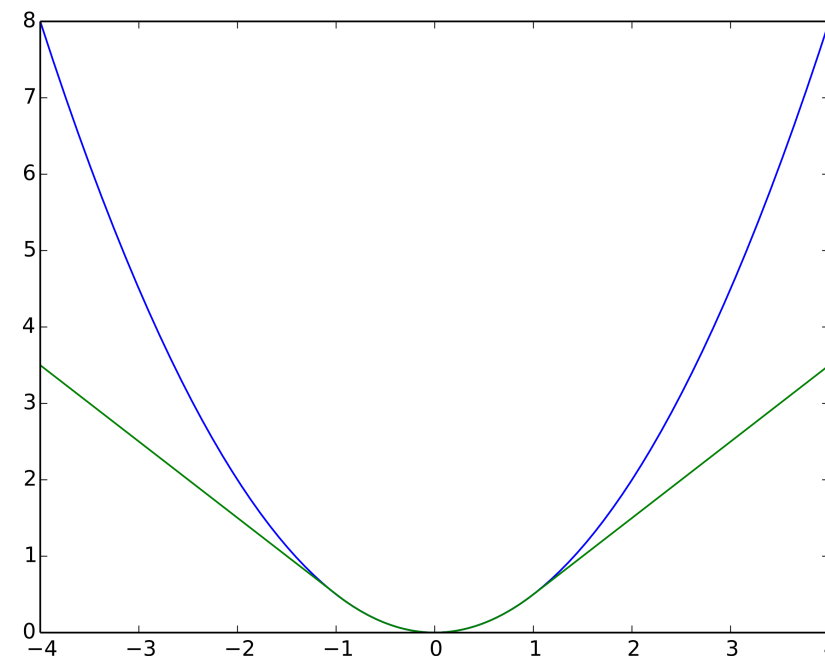
- Training instability due to non-stationary target y
- Training examples are highly correlated
 - Neural networks are prone to “catastrophic forgetting”
- Training instability due to very large positive or negative rewards

DQNs: Issues that Arise

- Instability in target y
 - Solution: Maintain a “target network” with weights θ^-
- Training examples are highly correlated
 - Solution: do experience replay
 - Keep all of your experience in a replay buffer
 - Each element is a tuple (s, a, r, s')
 - All that is needed to compute $y = r + \gamma \max_{a'} q_{\theta^-}(s', a')$
 - During training, randomly sample a batch from your replay buffer
 - Decorrelates training examples

DQNs: Huber Loss

- Training instability due to very large positive or negative rewards
- Clips loss after reaching a certain magnitude

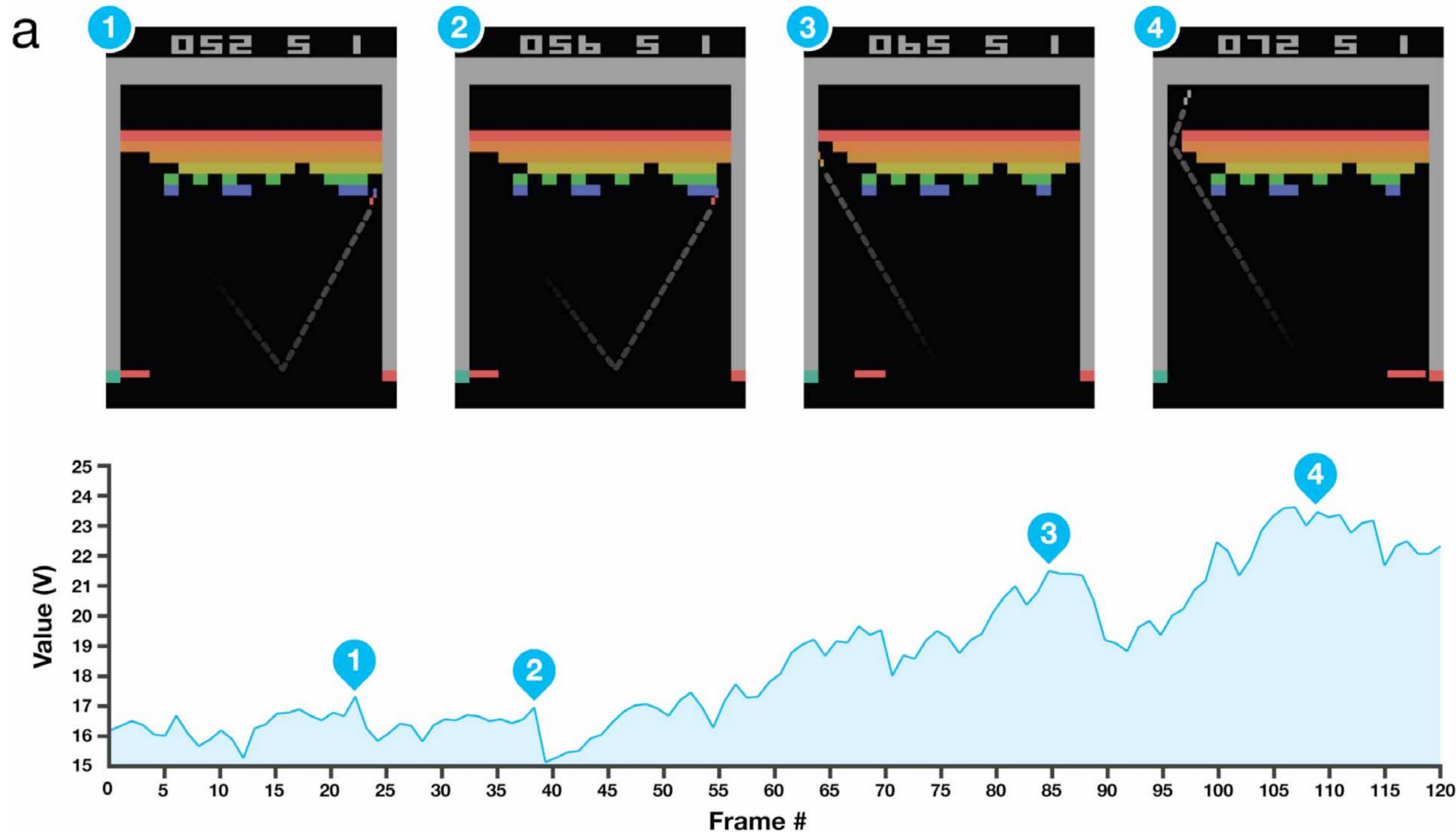


DQNs: Example on Atari



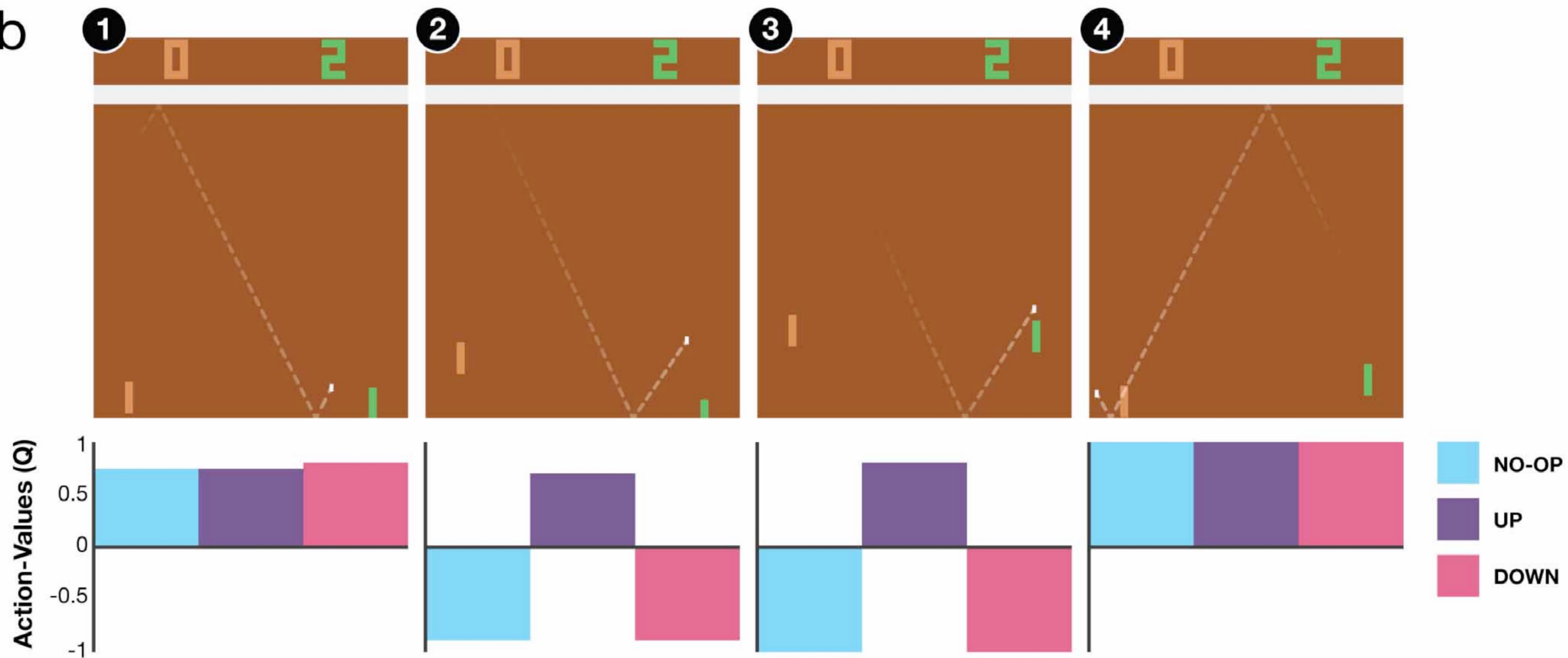
DQN: State Value Behavior

- Value fluctuation due to discounting as well as nature of imperfect function approximation and discounting
- How to get the state value function from the action-value function?

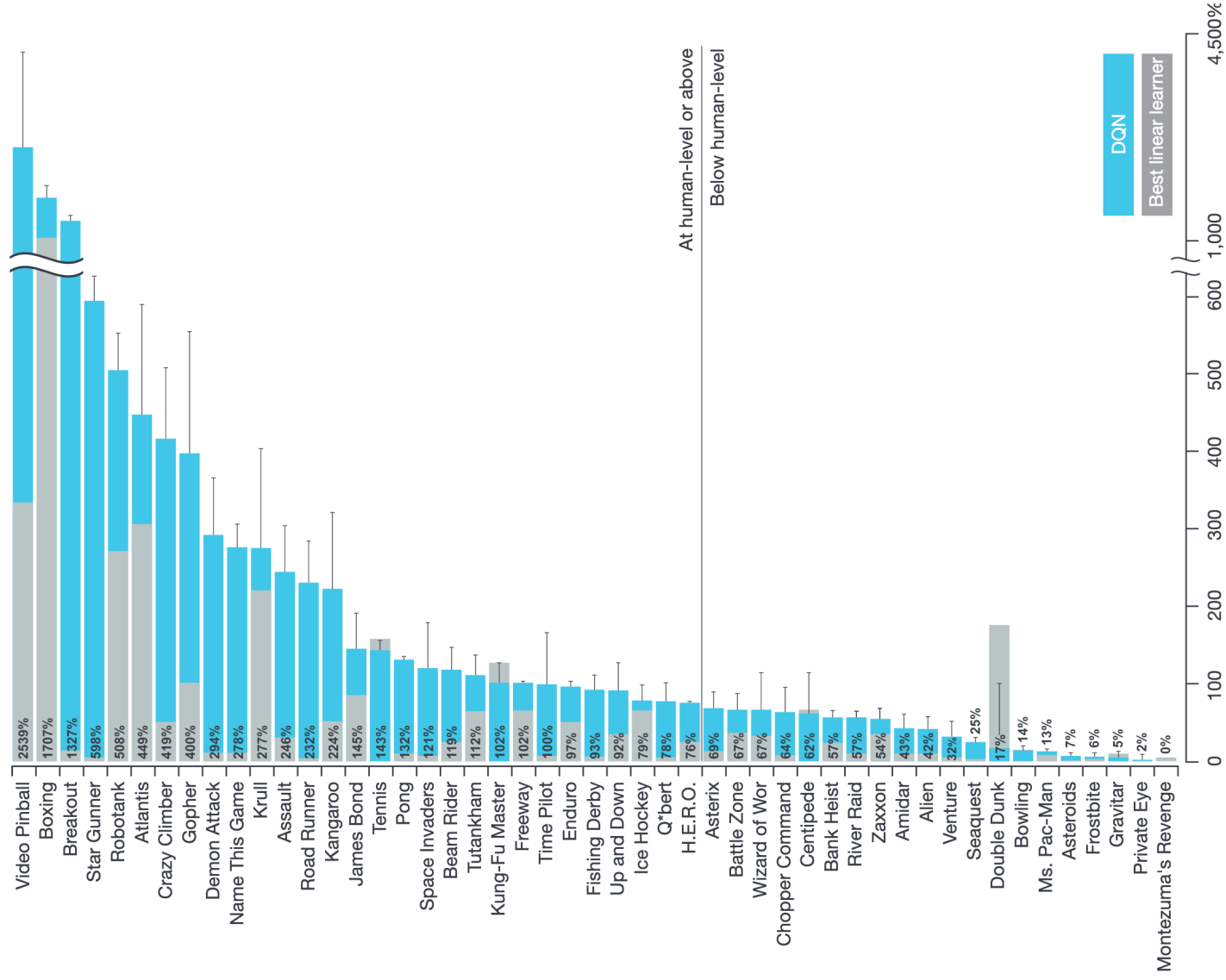


DQN: Action-Value Behavior

b



DQN: Performance on Atari



DQN: Ablation Study

Game	With replay, with target Q	With replay, without target Q	Without replay, with target Q	Without replay, without target Q
Breakout	316.8	240.7	10.2	3.2
Enduro	1006.3	831.4	141.9	29.1
River Raid	7446.6	4102.8	2867.7	1453.0
Seaquest	2894.4	822.6	1003.0	275.8
Space Invaders	1088.9	826.3	373.2	302.0

DQN: Ablation Study

- How do deep neural networks compare with a linear function approximator?

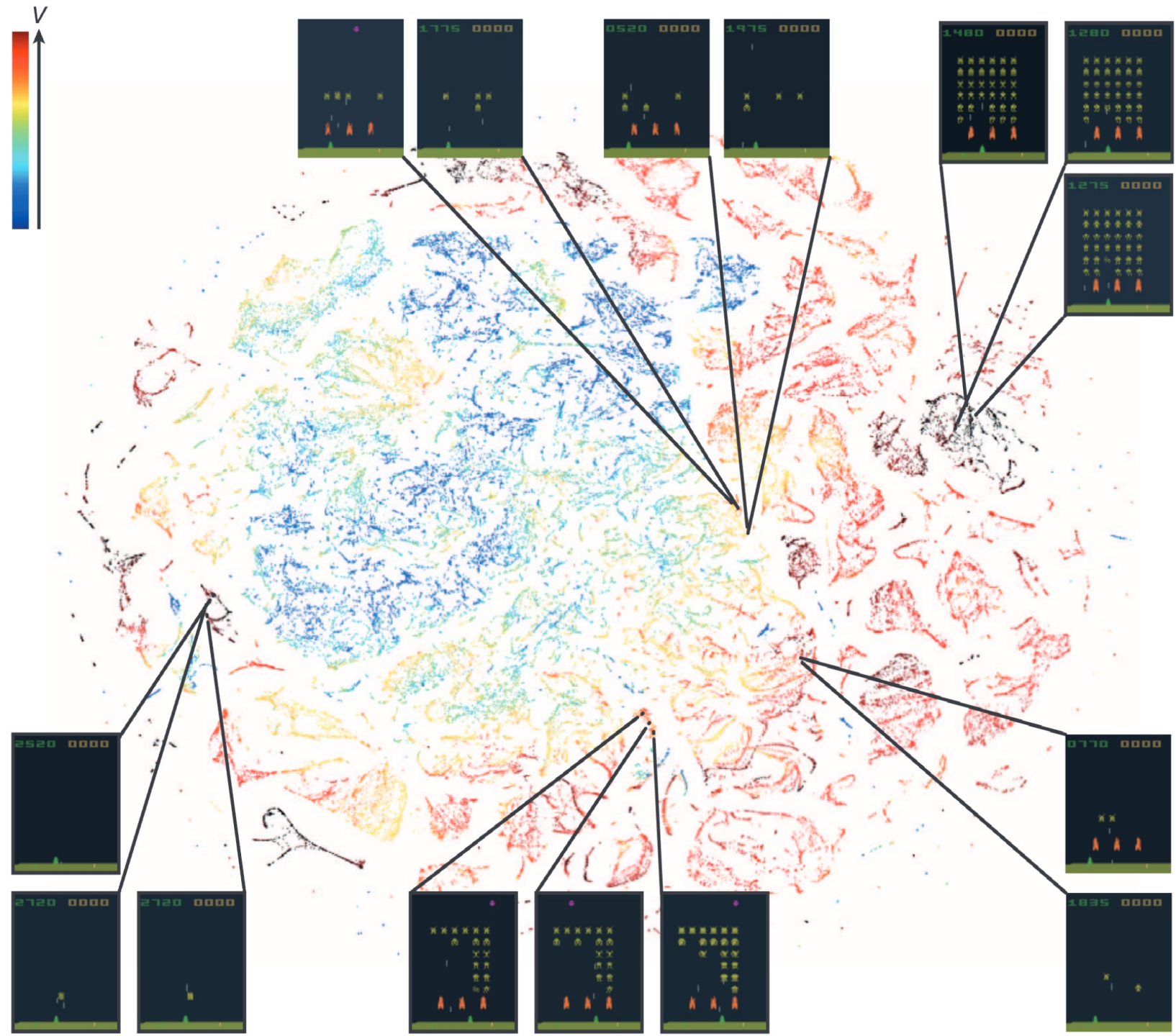
Game	DQN	Linear
Breakout	316.8	3.00
Enduro	1006.3	62.0
River Raid	7446.6	2346.9
Seaquest	2894.4	656.9
Space Invaders	1088.9	301.3

DQN: Hyperparameters

Hyperparameter	Value	Description
minibatch size	32	Number of training cases over which each stochastic gradient descent (SGD) update is computed.
replay memory size	1000000	SGD updates are sampled from this number of most recent frames.
agent history length	4	The number of most recent frames experienced by the agent that are given as input to the Q network.
target network update frequency	10000	The frequency (measured in the number of parameter updates) with which the target network is updated (this corresponds to the parameter C from Algorithm 1).
discount factor	0.99	Discount factor γ used in the Q-learning update.
action repeat	4	Repeat each action selected by the agent this many times. Using a value of 4 results in the agent seeing only every 4th input frame.
update frequency	4	The number of actions selected by the agent between successive SGD updates. Using a value of 4 results in the agent selecting 4 actions between each pair of successive updates.
learning rate	0.00025	The learning rate used by RMSProp.
gradient momentum	0.95	Gradient momentum used by RMSProp.
squared gradient momentum	0.95	Squared gradient (denominator) momentum used by RMSProp.
min squared gradient	0.01	Constant added to the squared gradient in the denominator of the RMSProp update.
initial exploration	1	Initial value of ϵ in ϵ -greedy exploration.
final exploration	0.1	Final value of ϵ in ϵ -greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ϵ is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of "do nothing" actions to be performed by the agent at the start of an episode.

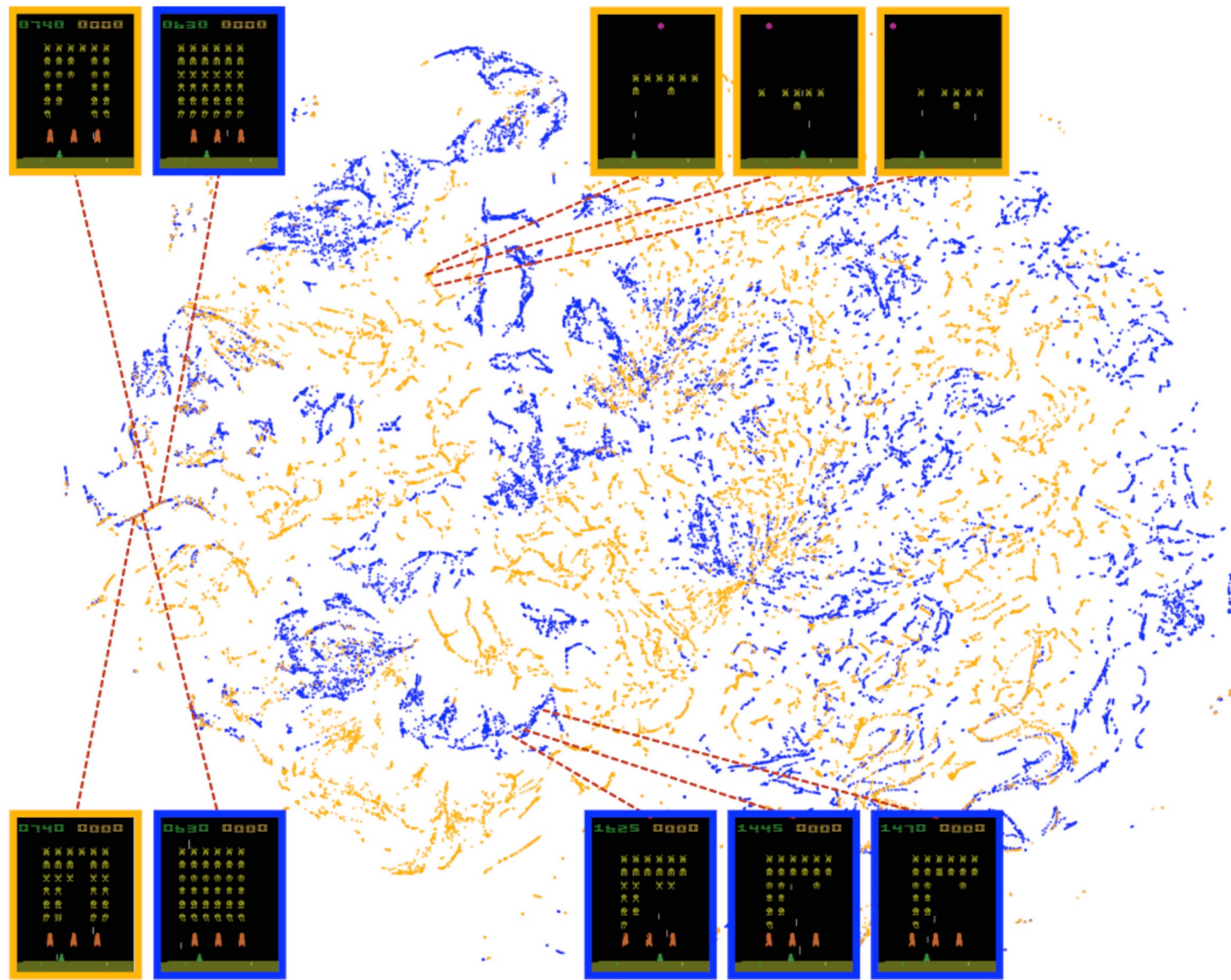
DQN: Last Hidden Layer Activations

- Done for space invaders
- Two-dimensional projection using t-SNE
- Semantically similar states are clustered next to each other



DQN: Last Hidden Layer Activations

- How does network generalize to states obtained under other policies?
 - Human: Orange
 - AI Agent: Blue

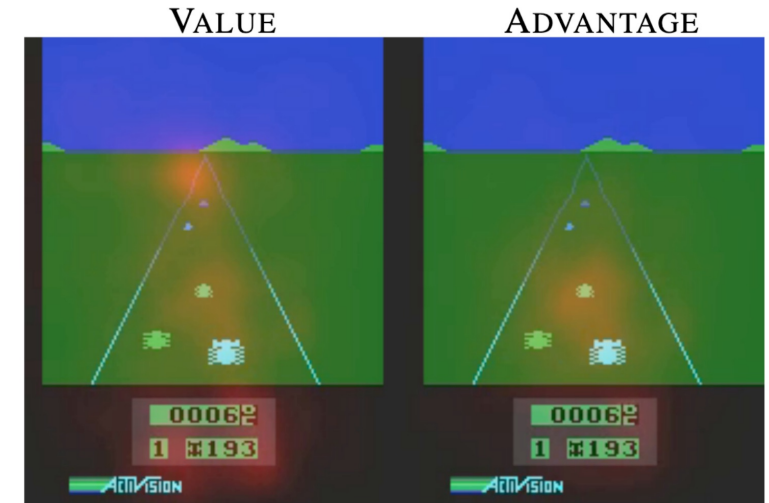
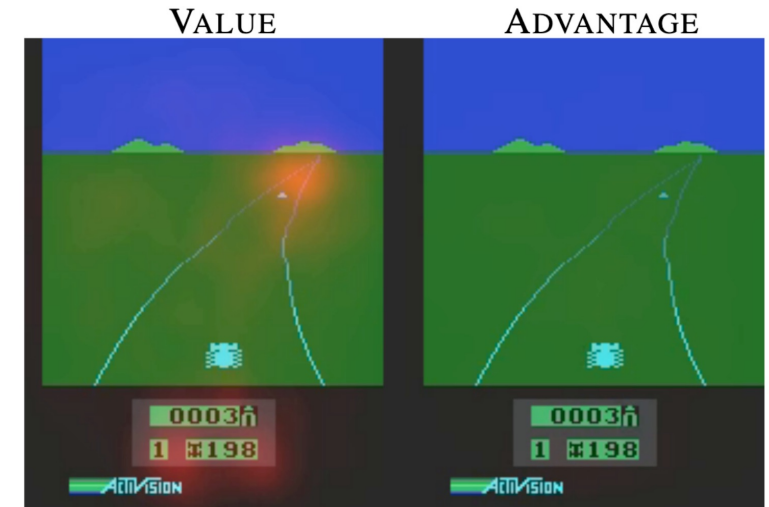
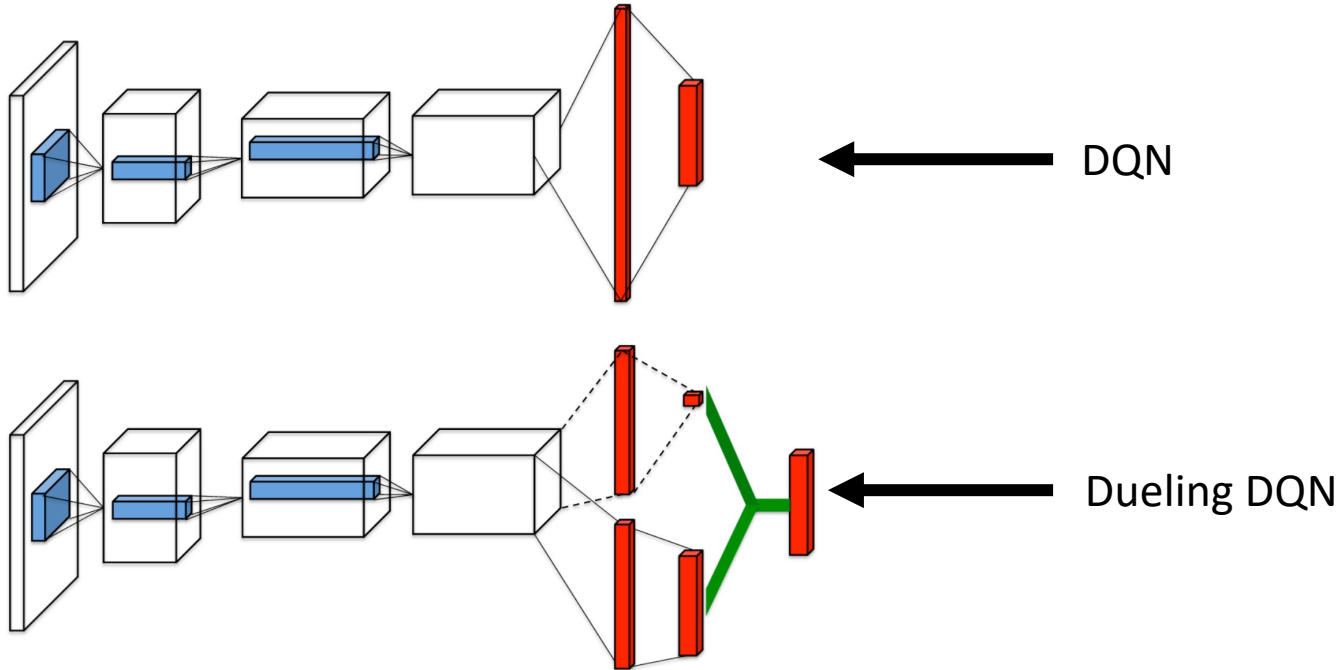


DQN Improvements: Prioritized Experience Replay

- Prioritize transitions in replay buffer with large temporal difference errors
 - $y =$
 - $r + \gamma \max_{a'} q_{\theta}(s', a')$ if s is not a terminal state
 - 0 otherwise
 - $L(\theta) = \frac{1}{2} (y - q_{\theta}(s, a))^2$

DQN Improvements: Dueling DQNs

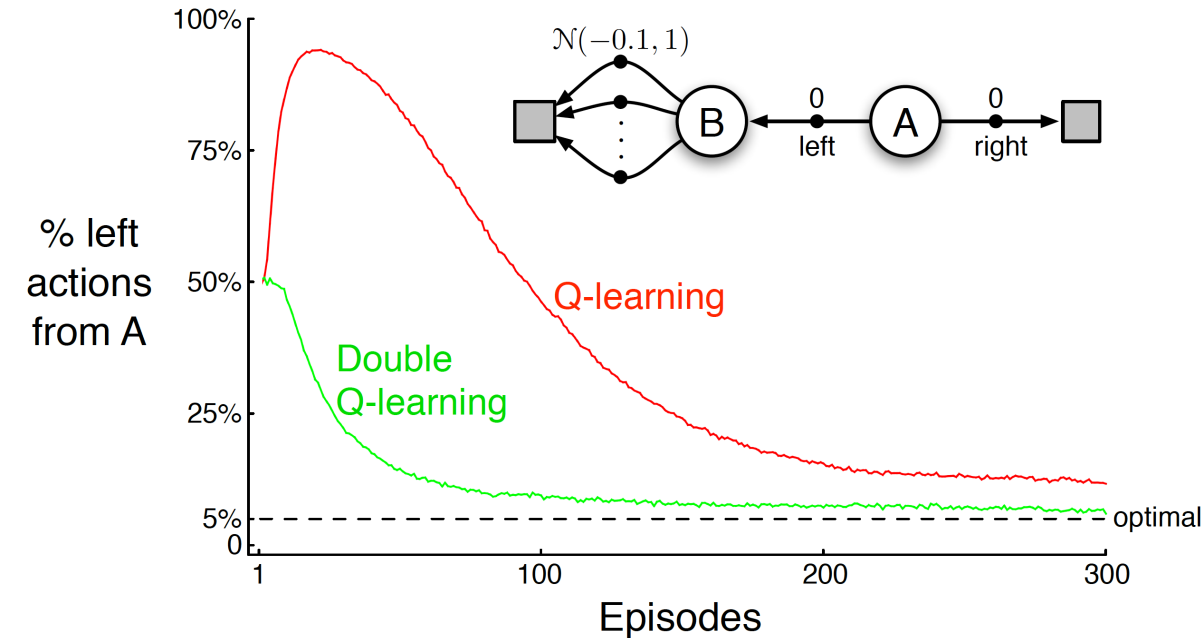
- Advantage function: $A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$
- Dueling DQN: $\hat{q}(s, a) = v_{\theta_v}(s) + A_{\theta_A}(s, a)$



DQN Improvements: Double Q-learning

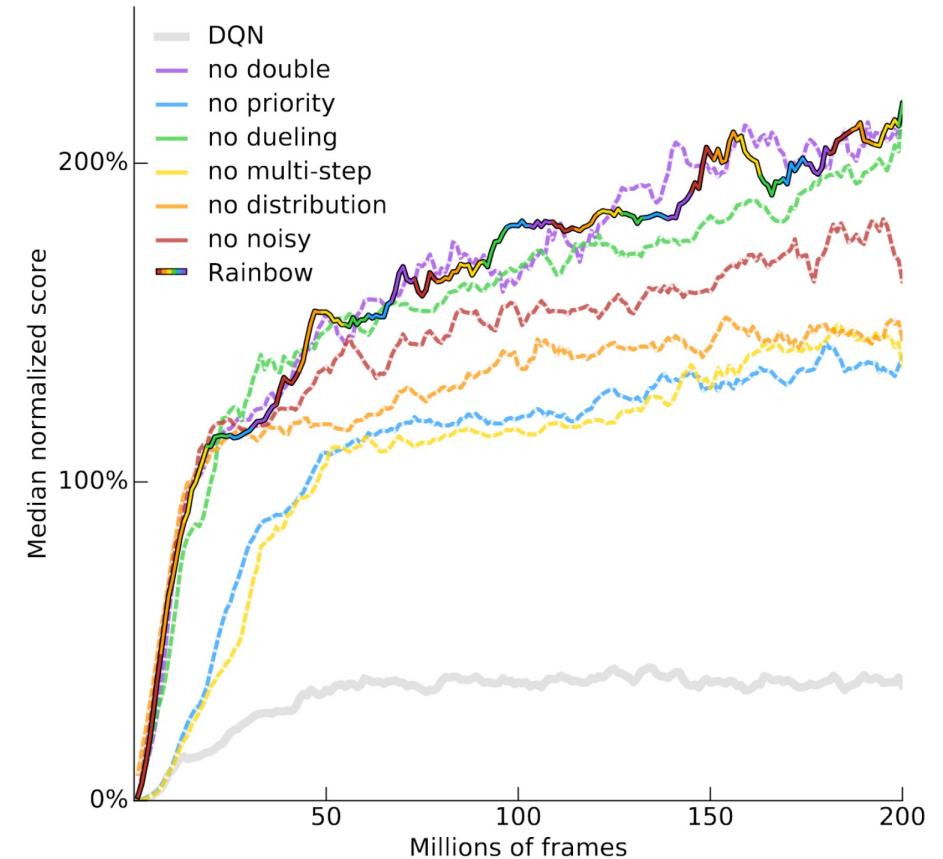
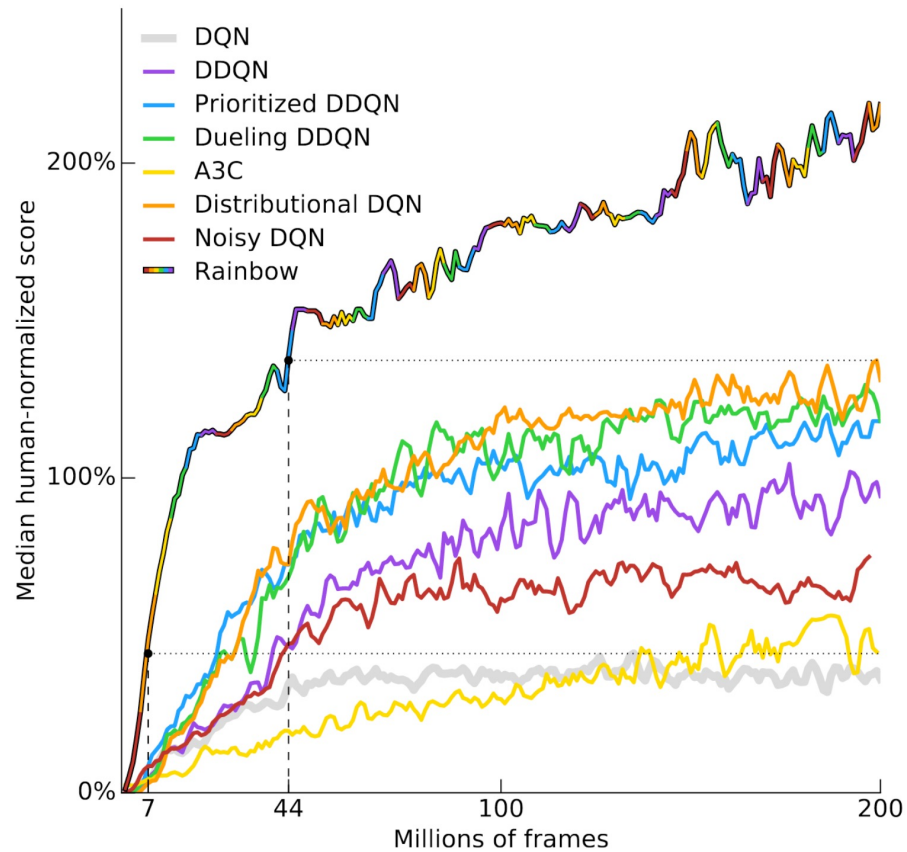
- Q-learning maximizes over actions to estimate the future reward
- This leads to a maximization bias effect
- We can solve this by using two Q-functions

- $y = r + \gamma \hat{q}_2(s', \arg\max_{a'} \hat{q}_1(s', a', \mathbf{w}), \mathbf{w})$
- $E(\mathbf{w}) = \frac{1}{2} (y - \hat{q}_1(s, a, \mathbf{w}))^2$



DQN Improvements: Rainbow

- Combines many of the DQN improvements



DQN Improvements: Agent 57

- First algorithm to outperform humans on all 57 games in the Atari benchmark
- Uses artificial curiosity
 - Intrinsic reward and extrinsic reward
- Adjusts discount factor
- Uses recurrent neural networks